FloraSense AI

Complete Deep Learning Project Report

Plant species identification and crop disease classification using transfer learning

Prepared from the project notebook, saved checkpoints, manifests, reports, and measured experiment outputs.

Project	FloraSense AI
Primary framework	PyTorch + Torchvision
Main model	MobileNetV3-Large
Baseline model	ResNet18
Input size	224 × 224 RGB image
Recorded runtime	Python 3.13.1, CPU

Executive Summary

FloraSense AI is a computer-vision system that analyzes leaf images. It contains three independent classifiers: a 10-class Campus plant model, a 12-class Kaggle species model, and a 38-class PlantVillage crop-condition model. The models use ImageNet-pretrained MobileNetV3-Large and ResNet18 networks, OpenCV preprocessing, cross-entropy loss, Adam optimization, validation macro-F1 checkpoint selection, and held-out test evaluation.

The saved experiments report test accuracy between 99.065% and 99.767%. MobileNetV3-Large is the recommended deployment model because its checkpoints are about 16 MB and it has lower CPU latency than ResNet18. ResNet18 provides slightly higher measured test accuracy in the current experiments, but it is about 43 MB and is slower in the recorded benchmarks.

Important interpretation: these results describe performance on the project's test sets. Real-world field performance requires additional validation using images from different phones, backgrounds, locations, cultivars, and disease stages.

Project goals

• Identify supported plant species from leaf images.

• Predict candidate crop disease classes using PlantVillage labels.

• Provide top-k predictions, confidence scores, and uncertainty warnings.

• Compare accuracy and deployment efficiency of two CNN architectures.

• Preserve reproducibility through manifests, class mappings, checkpoints, and environment metadata.

1. Problem Definition and Scope

Manual plant identification requires botanical expertise, while disease recognition can be difficult when symptoms are subtle or visually similar. FloraSense AI frames the task as supervised multi-class image classification.

Input	Output
One leaf or plant image	Top-k plant species predictions
Leaf image from a supported crop	Candidate disease or healthy condition
Prediction score	Confidence-aware status and warning
Recognized species	Verified care profile when available

The three taxonomies remain separate. A species label from the Kaggle dataset is not silently merged with a Campus label, and a PlantVillage disease prediction is not automatically treated as compatible with an arbitrary species prediction.

Project limitations

• The classifier only knows classes present in its training labels.

• A softmax score is not automatically a calibrated probability.

• A PlantVillage output is a candidate visual classification, not a confirmed diagnosis.

• A care profile is returned only when manually entered and source-grounded.

Example input images

Arjun.JPG: project input image

Mango.jpg: project input image

2. System Architecture

The architecture is a modular inference pipeline. Image decoding and preprocessing are shared, while the classification heads remain independent for each label vocabulary.

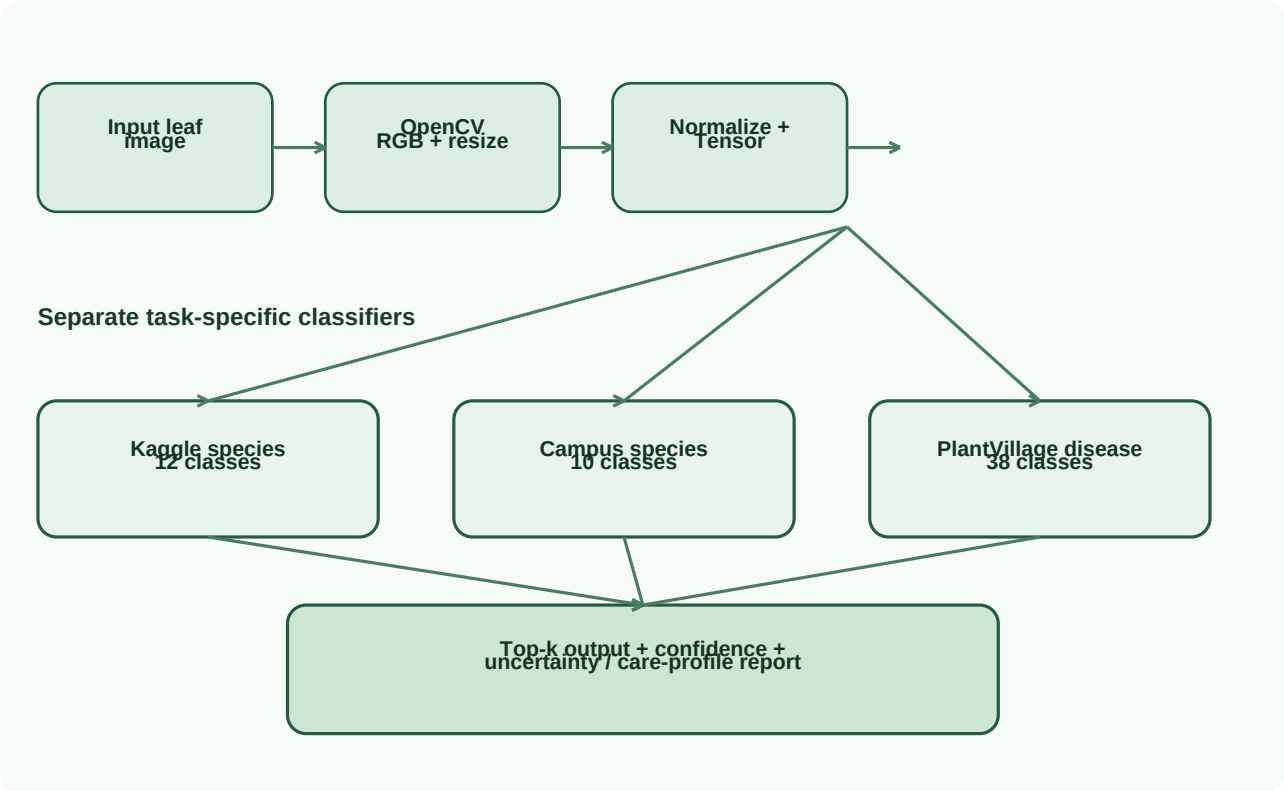

Figure 1. FloraSense AI end-to-end architecture.

Architectural principle

The system uses task-specific models rather than one universal classifier. This prevents incompatible class indices from being mixed and allows each dataset to retain its own label semantics.

Inference report

The final report can contain a species result from the supported species models, a separate PlantVillage candidate, top-k alternatives, a threshold-based uncertainty status, and a care profile lookup.

3. Dataset and Data Engineering Pipeline

Dataset/task	Classes	Use
Campus species	10	Common campus/garden plant identification
Kaggle species	12	Indian plant species identification
PlantVillage disease	38	Crop disease and healthy-condition classification

Dataset pipeline

• Inspect dataset roots and class directories.

• Find image files with supported extensions.

• Reject unreadable images using OpenCV.

• Create a manifest containing path, label, dataset, and split metadata.

• Preserve Campus TRAIN/VALID/TEST folders when available.

• Create stratified 70:15:15 train/validation/test splits for class-root datasets.

• Generate and save a deterministic class-to-index mapping.

Data leakage consideration

If multiple photographs show the same physical leaf or plant, an image-level random split may place related images in both training and test data. A production-quality collection should split by plant identity, leaf identity, capture session, or location.

Preprocessing

OpenCV decodes BGR images, converts them to RGB, and resizes them to 224 × 224. Training uses random rotation, color jitter, and horizontal flips. Validation and test images use deterministic tensor conversion and ImageNet normalization.

4. Deep Learning Concepts

Convolutional neural networks learn hierarchical visual features. Early layers learn edges and color boundaries; middle layers learn veins, textures, spots, and margins; later layers combine these patterns into species or disease evidence.

Concept	How it is used
Supervised learning	Images are paired with known class labels.
Multi-class classification	Each model selects one class from its own vocabulary.
Transfer learning	ImageNet-pretrained visual features are adapted to plant classes.
Cross-entropy loss	Penalizes low probability assigned to the correct class.
Adam optimizer	Updates model parameters using adaptive learning rates.
Weight decay	Regularizes parameters and helps reduce overfitting.
Data augmentation	Improves robustness to orientation and lighting variation.
Early stopping	Stops when validation macro-F1 no longer improves.
Macro metrics	Gives every class equal importance in the summary.

MobileNetV3-Large

MobileNetV3-Large is the main model because it is designed for efficient inference. It uses lightweight convolutional building blocks, inverted residual structures, and channel-attention mechanisms. It is well suited to mobile, CPU, and edge deployment.

ResNet18

ResNet18 is the comparison baseline. Its residual connections learn a transformation F(x) and add the original input x, helping optimization in deeper networks.

5. Training Workflow and Code Explanation

The notebook sets random seeds, selects CPU or GPU, builds manifests, creates Dataset and DataLoader objects, loads a pretrained network, replaces its final layer, and trains using mini-batch gradient descent.

Core training sequence

Stage	Operation
Forward pass	logits = model(images)
Loss	loss = CrossEntropyLoss(logits, labels)
Backpropagation	loss.backward()
Parameter update	optimizer.step()
Validation	Evaluate without updating weights
Checkpointing	Save the state with best validation macro-F1

The saved checkpoint includes architecture, model weights, class names, class mapping, image size, normalization statistics, best validation macro-F1, and seed. Keeping these fields together prevents incorrect labels or preprocessing during later inference.

Hyperparameters recorded in the environment

Parameter	Value
Image size	224 × 224
Batch size	32
Learning rate	0.0001
Weight decay	0.0001
Maximum epochs	10 in the recorded notebook configuration
Early stopping patience	4 epochs
Random seed	42

6. Evaluation Metrics

Accuracy is the fraction of all predictions that are correct. **Precision** asks how often a predicted class is correct. **Recall** asks how many actual examples of a class were found. **F1-score** is the harmonic mean of precision and recall.

Formulas

Metric	Formula	Interpretation
Accuracy	(TP + TN) / all samples	Overall correctness
Precision	TP / (TP + FP)	Controls false positives
Recall	TP / (TP + FN)	Controls missed positives
F1-score	2PR / (P + R)	Balances precision and recall

Macro averaging calculates each class metric independently and averages the classes equally. This is important when the number of images differs between classes. The notebook also saves per-class classification reports and confusion matrices.

Operational metrics

Model size, batch-1 latency, and FPS are also measured. The recorded latency benchmark measures the forward pass on an already preprocessed 224 × 224 tensor, so it does not include image decoding or OpenCV preprocessing.

7. Measured Test Results

The following values are taken from the saved final test metrics. Percentages are shown exactly to three decimal places.

Task	Architecture	Images	Accuracy	Macro P	Macro R	Macro F1	Size
kaggle species	mobilenet v3 large	642	99.065%	99.088%	98.926%	98.993%	16.293 MB
kaggle species	resnet18	642	99.377%	99.503%	99.115%	99.296%	42.736 MB
plantvillage disease	mobilenet v3 large	8146	99.619%	99.451%	99.365%	99.405%	16.426 MB
plantvillage disease	resnet18	8146	99.767%	99.629%	99.731%	99.678%	42.789 MB
campus species	mobilenet v3 large	3634	99.725%	99.722%	99.735%	99.728%	16.283 MB
campus species	resnet18	3634	99.752%	99.762%	99.762%	99.762%	42.732 MB

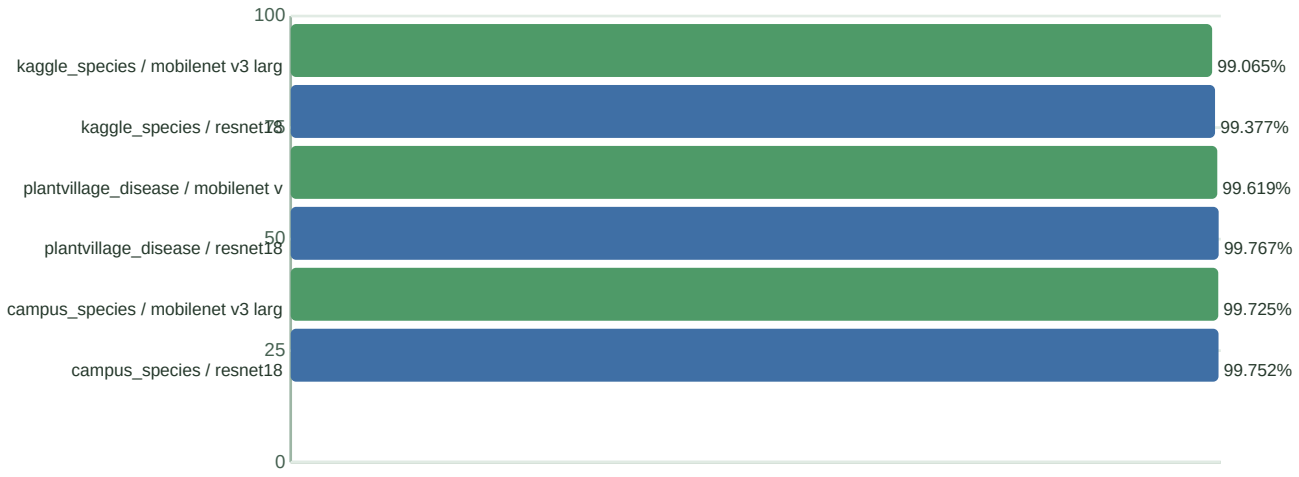

Figure 2. Held-out test accuracy by task and architecture.

Best recorded accuracy

- Kaggle species: ResNet18, 99.377% accuracy.
- PlantVillage disease: ResNet18, 99.767% accuracy.
- Campus species: ResNet18, 99.752% accuracy.

The results are excellent on the saved test sets, but they should not be interpreted as a guarantee of the same performance on uncontrolled field images.

8. Efficiency and Deployment Comparison

Task	Architecture	Latency	FPS	Size
kaggle species	mobilenet v3 large	13.424 ms	74.49	16.293 MB
kaggle species	resnet18	17.555 ms	56.96	42.736 MB
plantvillage disease	mobilenet v3 large	11.534 ms	86.70	16.426 MB
plantvillage disease	resnet18	69.380 ms	14.41	42.789 MB
campus species	mobilenet v3 large	12.218 ms	81.84	16.283 MB
campus species	resnet18	27.883 ms	35.86	42.732 MB

MobileNetV3-Large is the recommended deployment model because it is approximately 16 MB and generally gives higher CPU FPS. ResNet18 is useful when the small accuracy advantage is more important than memory and latency.

Recommended deployment choice

Requirement	Recommended model
Mobile or edge device	MobileNetV3-Large
Low memory and fast CPU inference	MobileNetV3-Large
Maximum current benchmark accuracy	ResNet18
Research baseline comparison	ResNet18

9. Inference, Confidence, and Care Workflow

At inference time, the image follows the same resize and normalization path used during evaluation. The model is put into evaluation mode, logits are converted with softmax, and the top-k classes are returned.

The notebook uses a practical confidence threshold of 0.40. If the highest score is lower than the threshold, the result is reported as uncertain and the user is advised to retake the image or seek expert confirmation.

Combined analysis must be interpreted carefully. A PlantVillage disease model should only be applied when the predicted species/crop is compatible with its crop label. Otherwise the disease result is merely an unsupported candidate.

Care profiles

The optional knowledge base stores common name, scientific name, description, soil, sunlight, watering, temperature, common problems, management notes, source URL, and review date. Unmatched labels return no profile rather than fabricated advice.

Suggested production response

Field	Example
Species	Mango
Confidence	0.96
Alternatives	Guava, Neem
Status	supported_prediction
Disease	Only if crop compatibility is confirmed
Warning	AI-assisted result; confirm with an expert

10. Benefits and Applications

• Agriculture: preliminary crop disease screening and field inspection assistance.

• Campus and gardens: plant inventory, biodiversity mapping, and educational identification.

• Nurseries: plant recognition and customer support.

• Environmental monitoring: local species cataloging and urban biodiversity surveys.

• Mobile AI: MobileNetV3-Large is small enough for offline and edge deployment.

• Education and research: demonstrates the full deep learning lifecycle from data preparation to deployment evaluation.

Limitations and risk controls

• Controlled datasets may not represent outdoor backgrounds and field conditions.

• Similar species and disease symptoms can be confused.

• Unsupported species should be reported as uncertain rather than forced into a known class.

• Disease predictions must not be treated as confirmed diagnoses.

• Care and chemical-treatment information must be verified against local agricultural guidance.

Recommended improvements

• Add a Streamlit, Gradio, or FastAPI user interface.

• Add leaf detection or segmentation for complex backgrounds.

• Collect real-world images across phones, regions, lighting, and disease stages.

• Use group-based dataset splitting to reduce leakage risk.

• Add Grad-CAM explanations and confidence calibration.

• Evaluate TorchScript, ONNX, quantization, and mobile deployment.

11. Conclusion

FloraSense AI implements a complete deep learning pipeline: dataset inspection, cleaning, manifest generation, stratified splitting, OpenCV preprocessing, augmentation, transfer learning, model training, validation, checkpoint selection, held-out testing, metric reporting, efficiency benchmarking, and image inference.

The saved test results show very high performance for the current datasets. MobileNetV3-Large is the most practical model for deployment because it offers a strong accuracy-efficiency balance. ResNet18 remains a useful accuracy baseline.

The next stage should focus on external validation. A deployment-ready system should be evaluated on real field images, include calibrated uncertainty, apply crop-compatible disease logic, and present the output as decision support rather than an unquestionable diagnosis.

Project artifact index

Artifact	Purpose
FloraSense_Final_Input_Based_DL.ipynb	Training, evaluation, inference, and benchmarking workflow
models/*.pth	Saved best model checkpoints
outputs/*_manifest.csv	Dataset paths, labels, and split records
outputs/*_class_mapping.json	Class-index lookup for inference
outputs/final_test_metrics.csv	Held-out test metrics
outputs/model_efficiency_metrics.csv	Latency, FPS, and checkpoint sizes
outputs/*_classification_report.txt	Per-class precision, recall, and F1

Source files used for this report: project notebook, saved model metrics, classification reports, environment metadata, class mappings, and the Arjun.JPG and Mango.jpg project images.